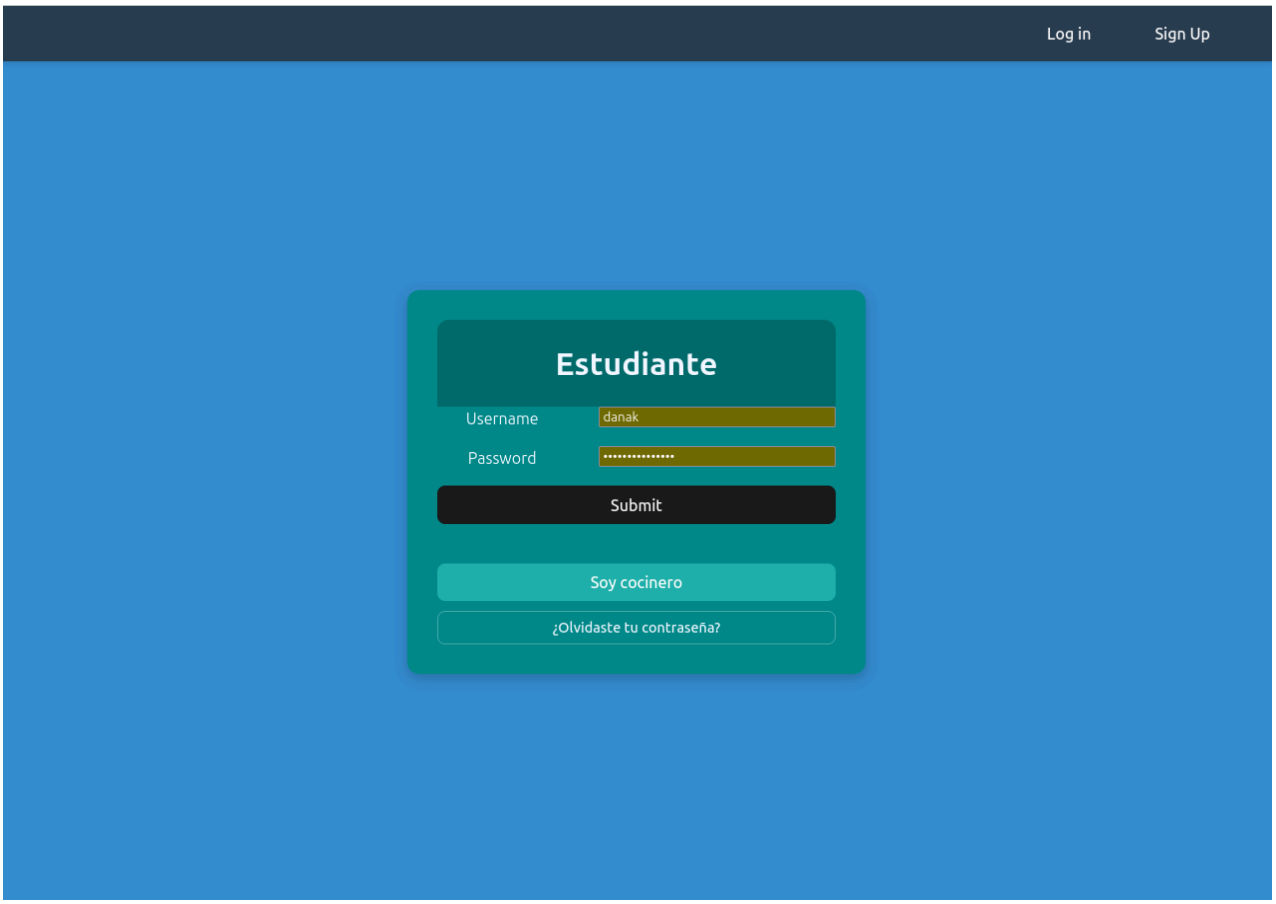
Reporte de pruebas Sprint 2:

Reestablecer contraseña:

Para hacer la prueba del funcionamiento correcto del reestablecimiento de una contraseña, con una cuenta creada, presiono en el botón que dice **¿Olvidaste tu contraseña?.**



Este botón nos redirige a una página donde debemos indicar el email con el que tenemos registrada nuestra cuenta.

Al presionar en el botón, nos envía al mail un link para elegir la nueva contraseña.

Ingresamos al link y nos aparece la siguiente pantalla, donde escribimos nuestra nueva contraseña.

Presionamos en *Reestablecer contraseña* y luego nos redirige a la página de login, donde finalmente ingresamos al sistema con la contraseña reestablecida correctamente.

Menú:

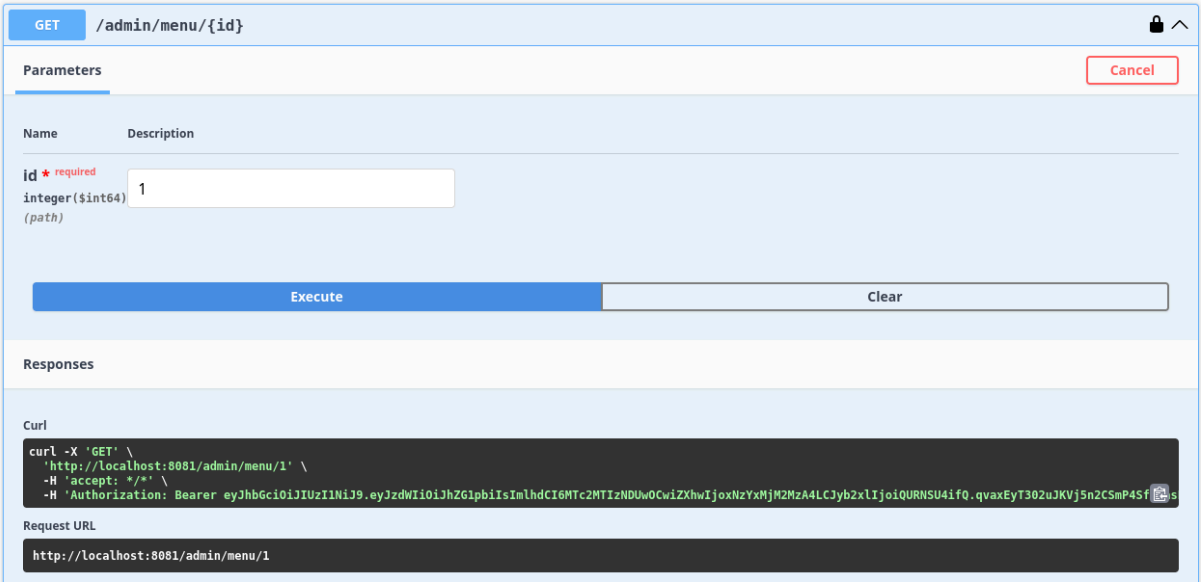
Se utilizaron los endpoints documentados en Swagger UI para validar el funcionamiento de los módulos de administración y menú.

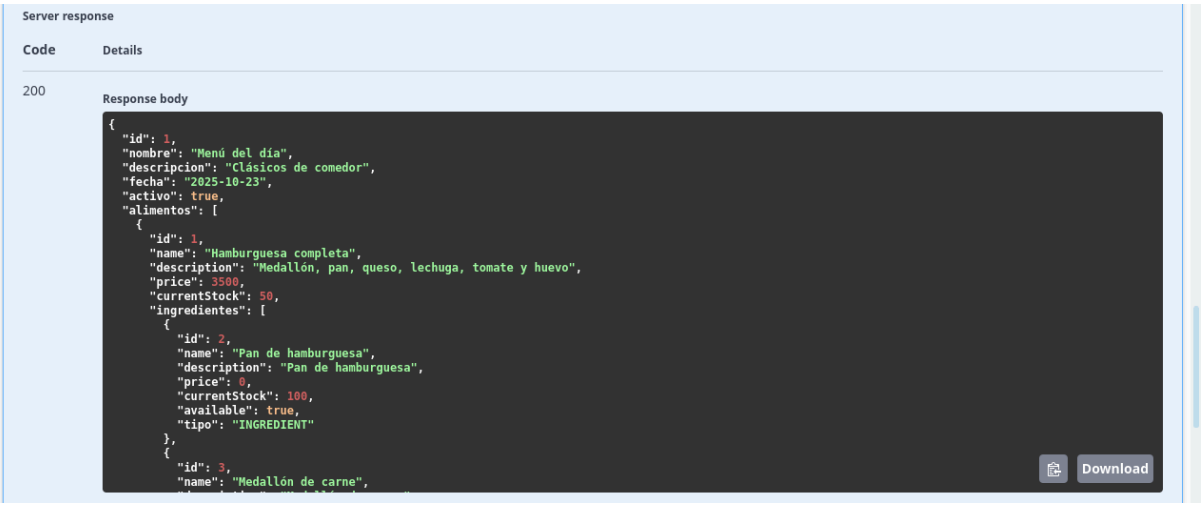
Como es el administrador el que maneja el menú, primero ingreso con ese rol.

<u>Casos de uso:</u>

1. Obtener un menú.

Realicé un GET menú con el ID 1 (el creado por default) y me lo devolvió. Contiene una lista de alimentos, que pueden ser productos o combos.





2. Crear un menú.

   Le pongo un nombre, descripción y fecha (no repetida).

POST /admin/menu

Parameters

No parameters

Request body required

```
{
  "nombre": "menú",
  "descripcion": "martes",
  "fecha": "2025-10-20"
}
```

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8081/admin/menu' \
  -H 'accept: */*' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJhZG1pbiIsImlhdCI6MTc2MTIzNjQ4NCwiZXhwIjoxNzYxMjM4Mjg0LCJyb2xlIjoiQURNSU4ifQ.rm53iGNThdVBh-6k0vgfb9z7p-nbd
  -H 'Content-Type: application/json' \
  -d '{
  "nombre": "menú",
  "descripcion": "martes",
  "fecha": "2025-10-20"
}'
```

Request URL

```
http://localhost:8081/admin/menu
```

Server response

Code	Details
201 *Undocumented*	Response body

```
{
  "id": 3,
  "nombre": "menú",
  "descripcion": "martes",
  "fecha": "2025-10-20",
  "activo": true,
  "alimentos": []
}
```

Download

Como vemos creó el menú con el id 3.

3. Agregar al menú un producto existente.

Al menú creado en el caso anterior le sumamos un producto existente, que pude obtener gracias al este GET:

Products

GET /products Obtener todos los productos

Parameters

Cancel

No parameters

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8081/products' \
  -H 'accept: */*' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJhZG1pbiIsImlhdCI6MTc2MTIyMzI3MSwiZXhwIjoxNzYxMjI1MDcxLCJyb2xlIjoiQURNSU4ifQ.z12tMsRFtaX8IefsrkEcCnhFQ
```

Request URL

```
http://localhost:8081/products
```

Server response

Code	Details
200	Response body


```
[
  {
    "id": 1,
    "name": "Hamburguesa completa",
    "description": "Medallón, pan, queso, lechuga, tomate y huevo",
    "price": 3500,
    "available": true,
    "currentStock": 50,
    "hasStock": true
  },
  {
    "id": 8,
    "name": "Milanesa con papas fritas",
    "description": "Milanesa de carne con guarnición de papas fritas",
    "price": 4200,
    "available": true,
    "currentStock": 40,
    "hasStock": true
  },
  {
    "id": 13,
    "name": "Pancho",
    "description": "Pan para pancho y salchicha",
    "price": 1800,
    "available": true,
    "currentStock": 80,
```


Download

Le agrego el producto con id 8 con la llamada POST al menú 3.

POST /admin/menu/{id}/productos/{productId}

Parameters

Cancel

Name	Description
id * required integer($int64) (path)	3
productId * required integer($int64) (path)	8

Execute | Clear

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8081/admin/menu/3/productos/8' \
  -H 'accept: */*' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJhZG1pbiIsImlhdCI6MTc2MTIzNjQ4NCwiZXhwIjoxNzYxMjM4Mjg0LCJyb2xlIjoiQURNSU4ifQ.rm53iGNThdVBh-6k0vgfb9z7p-nbd
  -d ''
```

Request URL

http://localhost:8081/admin/menu/3/productos/8

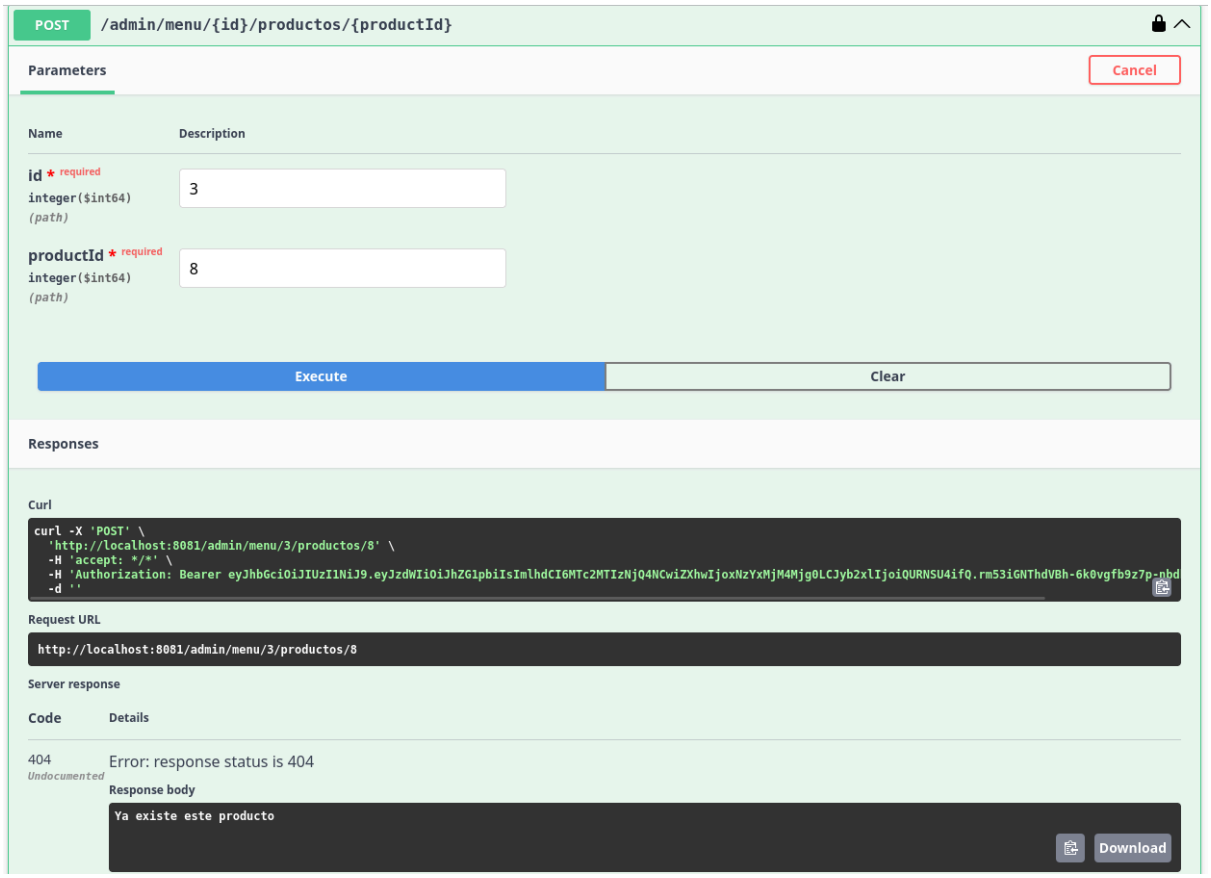
Server response

Code	Details
200	Response body


```
{
  "id": 3,
  "nombre": "menú",
  "descripcion": "martes",
  "fecha": "2025-10-20",
  "activo": true,
  "alimentos": [
    {
      "id": 8,
      "name": "Milanesa con papas fritas",
      "description": "Milanesa de carne con guarnición de papas fritas",
      "price": 4200,
      "currentStock": 40,
      "ingredientes": [
        {
          "id": 9,
          "name": "Milanesa",
          "description": "Milanesa",
          "price": 0,
          "currentStock": 60,
          "available": true,
          "tipo": "INGREDIENT"
        },
        {
          "id": 10,
          "name": "Papas fritas",
```

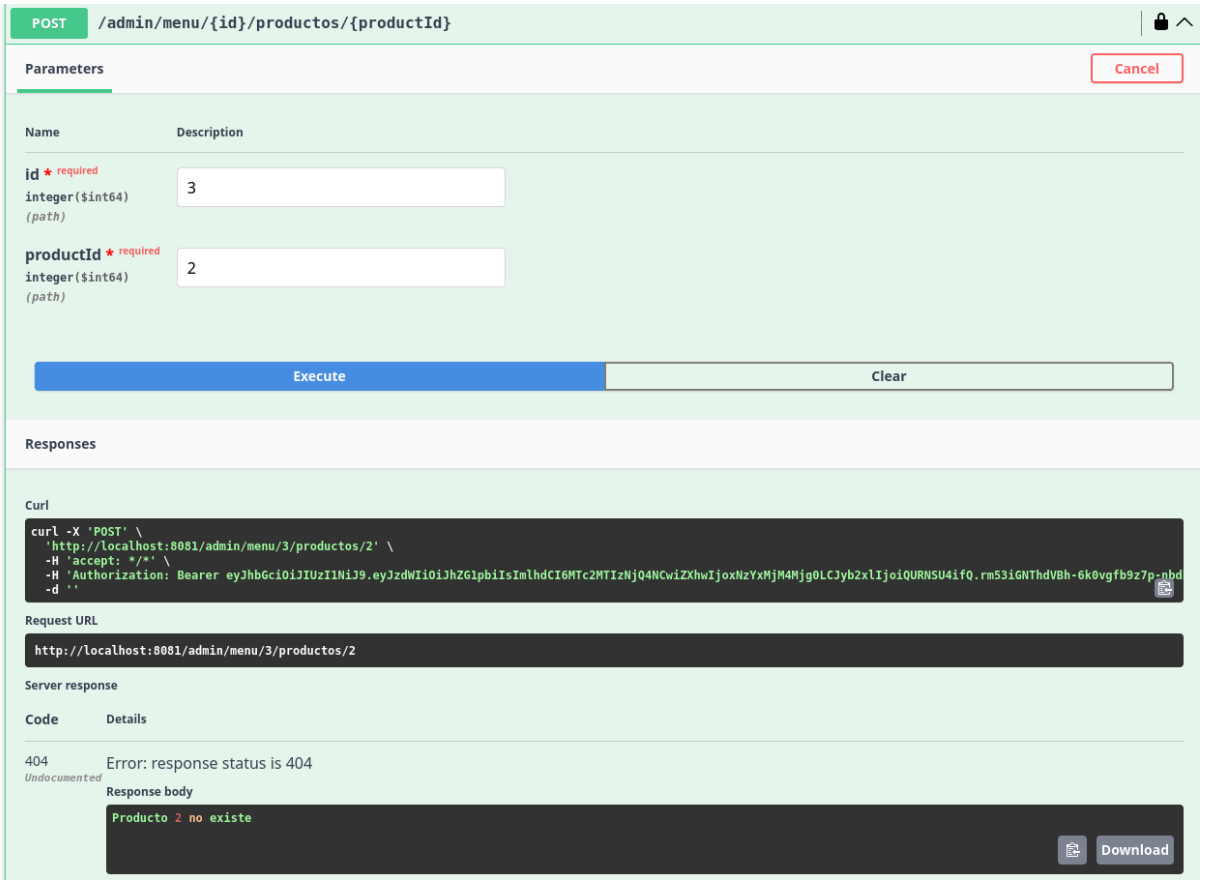

4. Error al intentar agregar producto duplicado.

Intento agregar nuevamente el producto 8 al menú 3.



5. Error al intentar agregar producto inexistente.

Ahora lanza un erro al intentar agregar el producto con el id 2 (que no existe) al menú 3.



Tests Unitarios:

Se realizaron pruebas unitarias sobre los servicios principales del sistema utilizando **JUnit 5 y Mockito**, para validar la lógica interna de negocio sin depender de la base de datos ni del servidor real. Verificamos la correcta creación, actualización y relación entre productos, ingredientes y combos.

Test Runner for Java

- addProductToCombo()
- createComboSuccessfully()
- createComboWithFewProductsThrows()
- deleteComboSuccess()
- deleteComboThrowsIfNotFound()
- getAllCombos()
- getComboById()
- removeProductThrowsIfOnlyTwoProducts()
- updateComboDiscount()

- createIngredientSuccessfully()
- decrementStockToZero()
- getIngredientById()
- incrementStock()
- ingredientNotFound()

- addIngredientProductNotFound()
- addIngredientToProduct()
- createProductSuccessfully()
- getProductById()
- productNotFound()
- updateProductSuccessfully()